



AI Agent for Lung Disease Detection

By

Muhammad Ehtisham Hanif
(ECI-IT-21-064)

Humair Hayat
(ECI-IT-21-058)

Faizan Rasool
(ECI-IT-21-057)

Under the supervision of

Miss Aisha Shafique

2025

Faculty of Engineering Sciences and Technology
Hamdard University Islamabad Campus, Pakistan



AI Agent for Lung Disease Detection

By

Muhammad Ehtisham Hanif
(ECI-IT-21-064)

Humair Hayat
(ECI-IT-21-058)

Faizan Rasool
(ECI-IT-21-057)

Under the supervision of

Miss Aisha Shafique

2025

Faculty of Engineering Sciences and Technology
Hamdard University Islamabad Campus, Pakistan

AI Agent for Lung Disease Detection

By

Muhammad Ehtisham Hanif
(ECI-IT-21-064)

Humair Hayat
(ECI-IT-21-058)

Faizan Rasool
(ECI-IT-21-057)

A project presented to the
Department Of Computing
In partial fulfillment of the
requirements for the degree of
Bachelors of Science
In
Computer Science

Faculty of Engineering Sciences and Technology
Hamdard University Islamabad Campus, Pakistan



Faculty of Engineering Sciences and Technology
Hamdard University Islamabad Campus, Pakistan

CERTIFICATE

This project “AI Agent for Lung Disease Detection” presented by **Muhammad Ehtisham Hanif, Humair Hayat, and Faizan Rasool** under the direction of their project advisor, **Miss Aisha Shafique**, and approved by the project examination committee, has been presented to and accepted by the Faculty of Engineering Sciences and Technology, in partial fulfillment of the requirements for **Bachelor of Science in Computer Science**.

Miss Aisha Shafique
(Project Advisor)

Dr. Muhammad Shaheer
(Examiner 1)

Sir Usman Javed
(Examiner 2)

Dr. Tahir Saleem
(Chairman, Department of Computing)

Dr. Hassan Raza
(Associate Dean, FEST)

Abstract

Tuberculosis (TB), COVID-19, and pneumonia are still major causes of illness and death worldwide. Chest X-rays are usually the easiest imaging test to get for first screening, but many regions have too few radiologists to read them. Deep learning can help with screening, yet most student and demo systems train a separate model for each disease, each on its own label set. That makes it impossible to compare architectures fairly, and a single uploaded image cannot get one answer covering all diseases. Reported accuracy is also often inflated by data leakage (the same patient or duplicate files in both training and test sets) and by preprocessing that differs between training and deployment.

We built an **AI Agent for Lung Disease Detection**, a chest X-ray system that sorts each image into one of four fixed classes: Normal, Tuberculosis, COVID-19, or Pneumonia. We trained three architectures on one shared manifest merged from three public datasets: EfficientNetB3 and DenseNet121, both fine-tuned from ImageNet, and a custom CNN trained from scratch. To keep the comparison fair, we removed duplicate images by content hash, split the data by patient, drew Normal and Pneumonia images from more than one source, capped class sizes and added inverse-frequency class weights, gave both transfer models the same classification head, and unfroze roughly the same share of each backbone (about 43%) during fine-tuning. We report per-class precision, recall, and F1-score, balanced accuracy, confusion matrices, and soft-voting ensemble metrics, and we ran a source-leakage probe, because overall accuracy alone hides how the minority class performs.

A Flask web application runs every available four-class model on each upload. It first checks that the image looks like a chest X-ray (the user can override this with “Analyze anyway”), then shows each model’s class probabilities, the ensemble verdict, and whether the models agree. Training and inference use the same bilinear resize. On the held-out test split ($n = 482$), DenseNet121 reached 90.87% accuracy and 89.14% balanced accuracy, and the soft-voting ensemble reached 92.32% accuracy and 93.07% balanced accuracy. COVID-19, the smallest class, had high recall but low precision. The source-leakage probe identified the source dataset with 88.17% accuracy against a 49.79% majority baseline, which indicates a moderate source signal. The system is a research demonstration built on public data and is **not** a clinical diagnostic device.

Acknowledgement

All praises and thanks to Al-Mighty “ALLAH”, the Most Merciful, the Most Gracious, the source of knowledge and wisdom endowed to mankind, who conferred us with the power of mind and capability to take this project to the exciting ocean of knowledge. All respects are for our most beloved Holy Prophet “Hazrat MUHAMMAD (Peace Be Upon Him)”, whose personality will always be a source of guidance for humanity.

We thank Hamdard Institute of Engineering and Technology for supporting this project.

We are grateful to our advisor, Miss Aisha Shafique, for her guidance and patience throughout the year. Her advice kept the project on track and pushed us to do better work.

Our classmates, and the three of us reviewing each other’s work, caught many problems early. Their comments and suggestions shaped the final system.

We thank our families and friends for their support and encouragement. We also thank the people who published the chest X-ray datasets this work depends on: the Kaggle tuberculosis and pneumonia datasets and the public COVID-19 radiography collections. Without them there would be no unified benchmark.

Project in Brief

Project Title	AI Agent for Lung Disease Detection
Objective	To develop a research-grade chest X-ray multi-disease classification system that benchmarks three deep learning architectures on a unified four-class dataset and deploys them through an ensemble-capable web application.
Main Contributions	• Fair 4-class architecture benchmark (shared manifest, head, and splits) • Ensemble CAD-style Flask demo with disagreement surfacing • Honest evaluation design (per-class metrics + leakage probe)
Limitation	Public small-data research demonstration only; not a clinical diagnostic device.
Undertaken By	• Muhammad Ehtisham Hanif (ECI-IT-21-064) • Humair Hayat (ECI-IT-21-058) • Faizan Rasool (ECI-IT-21-057)
Supervised By	Miss Aisha Shafique
Duration of Project	• Start Date: October 2024 • End Date: September 2025
Tools & Technologies Used	• Programming Languages: Python • Frameworks & Libraries: TensorFlow, Keras, Flask, OpenCV • Training pipeline: t r a i n i n g / (manifest, train, evaluate, leakage probe) • Development Tools: Jupyter Notebook, Visual Studio Code • Hardware: Standard CPU/GPU-based system used for training and testing
System Used	Windows-based development system using CPU/GPU processing for model training and testing.

Contents

Abstract	i
Acknowledgement	ii
Project in Brief	iv
1 Introduction	3
1.1 Motivation	3
1.2 Problem Statement	3
1.3 Aim and Objectives	4
1.4 Organization of the Report	4
2 Literature Review	6
2.1 Deep Learning in Medical Imaging	6
2.1.1 Foundational CNN Architectures	6
2.1.2 Large-Scale Chest X-Ray Benchmarks	7
2.2 COVID-19 Detection from Chest X-Rays	7
2.3 Tuberculosis Detection	9
2.4 Pneumonia Detection	10
2.5 Transfer Learning	10
2.6 Data Augmentation and Regularization	11
2.7 Multi-Disease Detection Frameworks	12
2.8 Deployment and Clinical Translation	12
2.9 Explainability and Trust	13
2.10 Gaps in Existing Research	14
2.11 Summary	14
3 Methodology	17
3.1 From Single-Disease Models to a Unified Benchmark	17
3.2 Data Collection and Unified Manifest	17
3.2.1 Source Datasets	18
3.2.2 Design Rules for Honest Splits	18
3.3 Preprocessing and Data Augmentation	19

3.3.1	Model-Specific Preprocessing	19
3.3.2	Augmentation and Class Imbalance	19
3.4	Model Architectures and Shared Head	20
3.5	Training Protocol	21
3.6	Evaluation Methodology	21
3.7	Application Integration Workflow	22
3.8	Methodological Validity and Limitations	22
3.9	Summary	23
4	System Design	25
4.1	System Architecture	25
4.2	Functional Requirements	25
4.3	Non-Functional Requirements	26
4.4	Tools and Technologies Used	26
4.5	Chest X-Ray Validation Gate	26
4.6	Application Flow and User Interface	27
4.7	Train/Serve Consistency	29
4.8	Summary	29
5	Results and Discussion	31
5.1	Experimental Setup	31
5.2	Unified Manifest Summary	31
5.3	Four-Class Comparative Results	32
5.4	Per-Class Precision, Recall, and F1	33
5.4.1	Recall summary	33
5.4.2	DenseNet121 (best single-model macro F1)	33
5.4.3	Ensemble (soft voting)	34
5.5	Confusion Matrices	34
5.6	Leakage Probe	35
5.7	Training Run Summary	35
5.8	Deployed Interface Demonstrations	36
5.9	Challenges Faced and Solutions Applied	38
5.10	Historical Single-Disease Work	38
5.11	Implementation Status	38
5.12	Summary of Findings	39
6	Future Recommendations	41
6.1	Conclusion	41

6.2	Recommendations	41
6.3	Future Enhancements	41
6.4	Summary of Contributions	41
References		42
A Dataset Details		45
A.1	Dataset Overview	45
A.1.1	Data Sources	45
A.1.2	Preprocessing and Splits	45
B Model Architecture Summaries		47
B.1	Model Training and Implementation	47
C Alignment with Sustainable Development Goals (SDGs)		49

List of Figures

Figure 3.1	End-to-end methodology pipeline for the unified four-class system	17
Figure 3.2	Dataset unification into a shared four-class training manifest	18
Figure 3.3	Preprocessing and augmentation pipeline with model-specific input sizes and normalisation	19
Figure 3.4	Three architectures scored under a matched four-class protocol (shared head for transfer models)	20
Figure 3.5	Two-phase transfer-learning protocol used for EfficientNetB3 and DenseNet121	21
Figure 3.6	Evaluation flow: matched test scoring, ensemble soft voting, and source- leakage probe	22
Figure 4.1	High-level inference flow for the unified four-class web application . . .	25
Figure 4.2	Three-stage chest X-ray validation gate with optional forced analysis . . .	27
Figure 4.3	Flask upload interface for four-class multi-model chest radiograph analysis	28
Figure 4.4	Chest X-ray validation gate rejection with Analyse anyway option	28
Figure 5.1	DenseNet121 confusion matrix	34
Figure 5.2	Ensemble confusion matrix	34
Figure 5.3	EfficientNetB3 confusion matrix	35
Figure 5.4	Custom CNN confusion matrix	35
Figure 5.5	Consensus Tuberculosis verdict with per-model 4-class probability bars .	36
Figure 5.6	Consensus COVID-19 verdict with per-model 4-class probability bars . .	37
Figure 5.7	Consensus Pneumonia verdict with per-model 4-class probability bars . .	37

List of Tables

Table 2.1	Selected related works versus this project’s focus	15
Table 3.1	Source datasets used to construct the unified four-class corpus	18
Table 3.2	Preprocessing per architecture	19
Table 3.3	Shared training hyperparameters and controls	21
Table 4.1	Model integration overview	25
Table 4.2	Flask application components	29
Table 5.1	Training environment (machine that produced the four-class weights) . . .	31
Table 5.2	Selected unified manifest counts by class and split	32
Table 5.3	Class $\times$ source contingency (selected manifest)	32
Table 5.4	Overall four-class test metrics ($n = 482$)	32
Table 5.5	Per-class recall on the test set	33
Table 5.6	DenseNet121 per-class metrics	33
Table 5.7	Ensemble per-class metrics	34
Table 5.8	Saved validation metrics and training cost	35
Table 5.9	Challenges and solutions during the 4-class conversion	38
Table 5.10	Implementation status after evaluation export	38
Table A.1	Dataset sources, paths, and known public links	45
Table C.1	Potential educational and research alignment with SDGs (non-clinical) . .	49

Chapter 1

Introduction

1.1 Motivation

Respiratory disease is a major global health problem. The World Health Organization lists lower respiratory infections among the ten leading causes of death worldwide, and tuberculosis alone causes more than 1.5 million deaths a year. Pneumonia, COVID-19, and other acute respiratory infections affect every age group, but children under five and elderly patients are the most likely to become seriously ill. The cost goes beyond deaths: hospital stays, lost work, and treatment of late-stage disease weigh most heavily on low- and middle-income countries, which also have the fewest diagnostic services.

Chest X-rays are usually the most available and cheapest imaging test for a first look at the lungs. They are quick to take, portable units can be used at the bedside, and they often show telltale signs such as consolidation, cavities, or ground-glass opacity. Reading them still needs a trained radiologist, and radiologists are unevenly spread. In many rural and high-burden areas, X-rays wait to be read, which delays triage, isolation, and treatment for TB, COVID-19, and pneumonia.

Convolutional neural networks (CNNs), especially ones fine-tuned from ImageNet, perform well on narrowly defined chest X-ray screening tasks. They learn visual features directly from the images, without hand-designed descriptors, and can flag suspicious cases for a radiologist to review first. Yet many student and demo systems still train one model per disease, each on its own label set. Those models cannot be compared fairly, and the user has to pick the suspected disease before seeing any result. Decisions such as which backbone to deploy, how to check uploads, and how to report recall on the rare class should rest on evidence.

This project therefore uses a single workflow: one set of four labels, three architectures trained under the same conditions, open reporting that includes per-class recall and a leakage probe, and one web application that shows every model's opinion along with an ensemble verdict. The system is meant to assist radiologists, and it is a reproducible research demonstration of multi-disease chest X-ray classification with honest evaluation and careful handling of uploads.

1.2 Problem Statement

Existing demo systems usually train separate models for separate diseases on incompatible label sets. The architectures cannot be compared fairly, and one upload cannot get a single assessment that covers all the diseases. Reported accuracy is often inflated by data leakage (the same patient or duplicate files across splits) and by preprocessing that differs between training and serving. Weak input checks can also let images that are not radiographs reach the models.

Problem: Design and implement a reproducible multi-architecture system that classifies chest X-rays as Normal, Tuberculosis, COVID-19, or Pneumonia using shared data and splits, reports its metrics openly, and offers a web interface that validates input and combines the models' predictions.

- Too few radiologists, and heavy workloads, delay chest X-ray reading in many places.
- Single-disease models trained on separate datasets cannot be compared, and they split the user interface into one tool per disease.
- A credible research system has to control leakage, use the same preprocessing in training and serving, and report performance on minority classes honestly, especially COVID-19, which has few samples.

1.3 Aim and Objectives

Aim:

- To develop a research-grade chest X-ray multi-disease classification system that benchmarks three deep learning architectures on a unified four-class dataset and deploys them through an ensemble-capable web application.

Objectives:

1. Merge three public CXR sources into a unified 4-class manifest with deduplication and patient-grouped splits.
2. Implement EfficientNetB3, DenseNet121, and a custom CNN with a matched training protocol and shared classification head (for transfer models).
3. Train and evaluate with per-class precision, recall, F1, balanced accuracy, confusion matrices, and a source-leakage probe.
4. Build a Flask application that runs all models per upload, computes a soft-voting ensemble, and applies a multi-stage chest X-ray gate.
5. Document methodology, limitations, and reproducibility (manifest, seeds, and scripts).

1.4 Organization of the Report

Chapter 2 reviews earlier work on deep learning for chest X-rays, transfer learning, multi-disease systems, and deployment, and identifies the gaps this project addresses. Chapter 3 covers the methodology: building the unified manifest, splitting the data without leakage, training with a shared head, evaluation, and integration into the application. Chapter 4 describes the system design, including its layers, requirements, radiograph gate, user interface, and how preprocessing is kept consistent between training and serving. Chapter 5 gives the experimental setup and results, the challenges we faced, earlier single-disease results (labelled as prior work), and implementation status. Chapter 6 gives conclusions, recommendations, and future work. The appendices list dataset details, model artefacts, and how the project relates to the Sustainable Development Goals.

Chapter 2

Literature Review

This chapter reviews published work on deep learning for chest X-ray analysis, focusing on COVID-19, tuberculosis, and pneumonia, the three diseases this project detects. It covers the main CNN architectures, transfer learning, the datasets in common use, multi-disease systems, and the gap between accuracy in the lab and tools that clinicians can use. Where a study shaped a decision in our system, we say so. The architecture studies led us to DenseNet121 and EfficientNetB3 alongside a CNN trained from scratch, the transfer learning work led to our two-phase training, the augmentation and regularization studies set our shared training settings, and criticism of undeployed research led to the Flask ensemble application described in Chapters 3 and 4.

2.1 Deep Learning in Medical Imaging

Medical image analysis has moved from hand-crafted features and classical machine learning to end-to-end deep learning. Older pipelines needed experts to define features such as texture descriptors, histogram statistics, and edge detectors, which were then passed to a classifier such as a support vector machine or random forest. A CNN learns its features directly from the pixels, so no manual feature engineering is needed, and on well-defined screening tasks its performance is getting close to that of specialists. This matters for chest X-rays because disease often shows up as small changes in brightness and texture across the lungs. Fixed hand-crafted descriptors capture such patterns poorly, while convolutional filters learn them well.

2.1.1 Foundational CNN Architectures

Most pulmonary imaging research builds on a handful of CNN designs. Simonyan and Zisserman (2014) introduced the VGG networks and showed that stacking many small 3×3 filters extracts features well. Using several 3×3 layers in place of one larger kernel adds non-linearity and depth to the receptive field without too many parameters. VGG16 and VGG19 are still common baselines for binary chest X-ray classification, especially TB screening, where very high accuracy has been reported on public datasets. Their simple structure also makes them easy to fine-tune: the final fully connected layers are replaced with a new classification head.

He et al. (2016) proposed ResNet, whose skip connections solve the degradation problem, in which adding layers to a very deep network makes training error go up. Gradients flow directly through the identity paths, so networks with hundreds of layers train stably. ResNet50 and ResNet50V2 appear often in medical imaging benchmarks as general-purpose backbones that combine depth with stable training.

Huang et al. (2017) introduced DenseNet, in which each layer receives the feature maps of every earlier layer in its dense block, not only the layer just before it. This improves gradient flow, lets layers reuse features, and needs fewer parameters for a given depth. DenseNet works especially well on chest X-rays, where faint texture and opacity patterns need to survive through

the network, which is why we chose DenseNet121 for the COVID-19 module.

Tan and Le (2019) proposed EfficientNet, which uses compound scaling to grow depth, width, and input resolution together by a fixed coefficient. Balancing all three, instead of enlarging one, gives good accuracy for the compute spent. EfficientNet suits deployments where inference time and memory matter, and those concerns led us to pick EfficientNetB3 for the tuberculosis module after benchmarking seven architectures.

2.1.2 Large-Scale Chest X-Ray Benchmarks

Deep learning for chest X-rays drew wide attention after CheXNet (Rajpurkar et al., 2017), a DenseNet121 model trained on ChestX-ray14 that detected pneumonia with radiologist-level accuracy on a held-out test set. CheXNet output a sigmoid probability of pneumonia and was trained on more than 100,000 frontal chest X-rays labelled with fourteen chest diseases. It made DenseNet a standard baseline for chest X-ray work and prompted a large body of follow-up research. It also showed that, on large datasets, deep learning can match specialists when the task is clearly defined and the evaluation is careful.

Irvin et al. (2019) released CheXpert, 224,316 chest X-rays labelled for 14 conditions by a rule-based labeler, with radiologists reviewing uncertain cases. Their DenseNet121 baseline matched radiologists on five conditions, including pneumonia and pleural effusion. It used an uncertainty-aware loss that handles ambiguous labels directly, instead of forcing them to positive or negative. CheXpert and similar large datasets showed that deep learning can be clinically useful for chest X-rays, provided that label noise, class imbalance, and differences between hospitals are handled through careful preprocessing and validation. CheXpert's size also shows that training one model on many conditions at once needs far more data than a student project has, which supports the modular single-task approach taken in this report.

These benchmark studies share three findings. First, almost all chest X-ray work starts from ImageNet weights, because medical datasets are rarely large enough to train deep networks from scratch without overfitting. Second, the choice of architecture affects both peak accuracy and inference cost, as our own TB benchmark showed when comparing VGG and EfficientNet. Third, the size and label quality of the dataset largely decide how well a model generalizes: high accuracy on a public Kaggle dataset does not carry over to images from another hospital or scanner without further validation. Our methodology follows from all three. We use transfer learning with phased fine-tuning for COVID-19 and TB, compare architectures systematically before choosing the TB model, and evaluate all three models on fixed held-out test splits.

2.2 COVID-19 Detection from Chest X-Rays

The COVID-19 pandemic sped up research into automated screening from portable chest X-rays. Emergency departments and poorly resourced hospitals needed fast triage when RT-PCR testing

was scarce or slow. Portable X-ray units could be taken to ICU beds and field hospitals and produce images within minutes, so automated reading offered another route to a result when test kits or laboratories were overwhelmed.

Wang et al. (2020) introduced COVID-Net, a custom CNN that classifies X-rays as COVID-19, non-COVID pneumonia, or normal, and reported sensitivity above 90%. They designed the architecture for the task, with depthwise separable convolutions and tailored filter banks, instead of relying only on standard ImageNet classifiers. Their results showed that COVID-19 opacities look similar to those of other pneumonias. That overlap remains a problem for multi-class screening and is one reason our system uses a single four-class label set.

Chowdhury et al. (2020) published the COVID-19 Radiography Database, one of the public sources merged into our manifest, and reported strong performance for COVID-19 versus normal on it. In our system these images supply the COVID-19 class, plus Normal and Pneumonia images where present. Images labelled “Viral Pneumonia” in the COVID sources go into the shared Pneumonia class, because the Kaggle pneumonia dataset mixes bacterial and viral cases and they cannot be separated again. The images are stored in one folder per class, so the manifest builder can load them by directory.

Rahman et al. (2021) studied how image enhancement and preprocessing affect COVID-19 detection and found that suitable contrast normalization and enhancement can raise performance by several percentage points. This supports using standard preprocessing matched to each model, which we do for all three models: `preprocess_input` for DenseNet and EfficientNet, division by 255 for the custom CNN, and a resize to each model’s expected input shape.

Zhang et al. (2020) introduced confidence-aware loss functions for COVID-19 detection that make predictions more reliable when labels are uncertain. Their model reached 96% sensitivity while keeping false positives under control. Screening tools need both properties: during an outbreak a false negative can let the virus keep spreading, while too many false positives can fill the available isolation beds.

Taken together, the COVID-19 literature shows that DenseNet-family models, multi-phase fine-tuning, and careful separation of COVID-19 from other opacities are central to useful multi-class screening. We use DenseNet121, from the same family as CheXNet (Rajpurkar et al., 2017), as one of the three architectures trained on the four-class task. Our two-phase training, with a frozen backbone followed by partial unfreezing, follows the gradual fine-tuning recommended by Raghu et al. (2019). Making COVID-19 one of four shared output classes, instead of a separate single-disease app, addresses the comparability problem described in Section 2.10. Chapter 3 describes the shared training code and Chapter 5 evaluates it.

2.3 Tuberculosis Detection

Tuberculosis is still one of the leading infectious causes of death. The World Health Organization (2022) reports more than 1.5 million TB deaths a year, and late diagnosis keeps the disease spreading, especially in high-burden regions with few radiologists. Chest X-rays are central to TB screening programmes, since upper-lobe infiltrates, cavities, and pleural effusion can suggest active disease before bacteriological tests confirm it. Automated TB detection from frontal chest X-rays is therefore a well-defined screening task with clear public health value.

Jaeger et al. (2014) set early TB detection benchmarks on the public Montgomery County and Shenzhen Hospital datasets, with standard evaluation splits that allowed algorithms to be compared reproducibly. Before these datasets, TB studies used inconsistent private data, so results across studies could not be compared reliably. The Montgomery and Shenzhen datasets are still reference points for TB screening research and influenced the later Kaggle TB datasets used in this project.

Lakhani and Sundaram (2017) applied an ensemble of GoogLeNet and AlexNet to chest X-ray datasets and reported AUC values of 0.99 and 0.97 respectively. Theirs was among the first studies to show that deep learning could match or beat radiologists at classifying TB from frontal chest X-rays. Their ensemble combined two architectures with different receptive fields, an early form of the multi-architecture comparison we carried out for the TB module.

Hwang et al. (2016) trained a CNN on more than 40,000 chest X-rays and reached an AUC of 0.96 for TB screening, before ImageNet transfer learning became common in medical imaging. Their work showed that with enough labelled examples, a larger dataset can make up for a simpler architecture.

More recent studies compare modern architectures on Kaggle TB datasets. Fine-tuned VGG models regularly reach very high accuracy on binary Normal versus Tuberculosis tasks, and EfficientNet models come close at a lower computational cost. Tan and Le (2019) showed that compound scaling lets EfficientNetB3 approach the accuracy of heavier architectures while running faster, which is why we included EfficientNetB3 in our four-class comparison alongside DenseNet121 and a custom CNN.

The TB literature supports two points we adopted. Architectures need to be benchmarked on the actual task, because the best model on an isolated binary task may not be the best on a harder shared multi-class problem. And binary TB screening on curated public datasets can reach near-perfect accuracy with transfer learning and regularization, but those figures cannot be compared with four-class results on patient-grouped splits. Real-world use would still need careful preprocessing, held-out evaluation, and prospective clinical validation.

2.4 Pneumonia Detection

CNN-based pneumonia detection has been benchmarked heavily on the Kaggle Chest X-Ray dataset released by Mooney (2018). It contains about 5,863 pediatric chest X-rays labelled NORMAL or PNEUMONIA, split into training, validation, and test folders. At that size overfitting is a known risk and regularization is needed. Because the splits are fixed, researchers can report results on the same held-out test set and compare architectures under identical conditions.

Studies on this dataset report accuracy above 90% with architectures ranging from small sequential CNNs to large transfer learning models. Rajpurkar et al. (2017) and later CheXpert-related work treat pneumonia as one of the most studied chest X-ray findings, with CNNs learning the opacity, consolidation, and interstitial changes seen in bacterial and viral infection. On frontal pediatric X-rays, pneumonia usually appears as increased opacity in one or both lungs, often with air bronchograms inside the consolidated areas, and convolutional filters can learn these patterns when there are enough labelled examples.

On this moderately sized dataset, batch normalization, dropout, and learning rate scheduling consistently improve generalization. Batch normalization keeps activations stable across mini-batches, dropout switches off random neurons during training so they cannot co-adapt, and reducing the learning rate when validation loss plateaus allows finer weight updates. Custom CNNs of shallow to medium depth remain competitive when training time, model size, and a simple architecture matter more than the small accuracy gain of a very deep transfer model. That trade-off led to the five-block custom CNN used as Model 3.

Rahman et al. (2021) also studied preprocessing for pneumonia and COVID-19 classification and found that pixel normalization, resizing method, and color channel order all affect inference. Channel order matters when images are loaded with OpenCV and converted between BGR and RGB, as our shared preprocessing code does for all models. When preprocessing at inference differs from preprocessing in training, deployed medical AI systems can lose accuracy without any visible error.

The Kaggle pneumonia dataset labels images only as NORMAL or PNEUMONIA, with no subtype or cause. We map those folders into the shared Normal and Pneumonia classes, alongside the TB and COVID sources, so that every architecture is compared on the same four outputs.

2.5 Transfer Learning

When labelled medical data is limited, the standard approach is transfer learning from models pre-trained on ImageNet, such as VGG16, InceptionV3, ResNet50, DenseNet121, and EfficientNet. ImageNet has over a million natural images in a thousand categories, and a model trained on it learns general low-level features such as edges, textures, and shapes. These carry over well to

X-rays, even though X-rays look very different from photographs. Medical datasets are rarely large enough to train deep networks from random weights without severe overfitting.

Raghu et al. (2019) studied transfer learning for medical imaging in their Transfusion paper. They found that low-level features transfer well, while higher-level features need fine-tuning for the new domain. They also observed that freezing the early layers and unfreezing deeper blocks gradually, or training a new classification head before fine-tuning the whole network, often beats training from random weights on small medical datasets. Keeping the backbone frozen at first stops large gradient updates from wrecking the pre-trained features while the new head is still random.

We follow this directly in the two-phase training of the DenseNet121 COVID-19 model. Phase 1 freezes the backbone and trains only the new head at a learning rate of 10^{-4} . Phase 2 unfreezes the backbone from layer 300 onward and fine-tunes at 10^{-5} . The EfficientNetB3 TB model works the same way: the ImageNet backbone starts frozen, and the classification head is trained with augmentation built into the model graph before the weights are saved for deployment.

The literature also warns that transfer learning does not guarantee generalization. Results on small public datasets can be inflated by leakage between training and test sets, by repeated tuning on the validation set, or by dataset artefacts such as hospital-specific borders or text that the model learns as a shortcut. Recommended practice is to fix random seeds, evaluate on a held-out test set, and compare architectures on identical splits, as we did in the TB benchmark with seed 1337.

2.6 Data Augmentation and Regularization

Medical imaging datasets are usually small compared with natural image datasets, so augmentation and regularization are needed to control overfitting. Augmentation enlarges the training set by applying transforms that do not change the label, which pushes the model to learn features that hold under those changes instead of memorizing pixels. For chest X-rays the literature favours random rotation and horizontal flips (frontal views are roughly symmetric left to right), width and height shifts that mimic small differences in patient position, zoom that mimics differences in scale, and other geometric transforms that leave intensities unchanged, applied only during training.

Dropout, batch normalization, early stopping, learning rate reduction on plateau, and checkpointing appear in most of the best-performing lung disease studies. Early stopping watches validation loss and stops training when it has not improved for a set number of epochs, so the model does not keep fitting noise. Checkpointing keeps the weights from the best epoch, since the final epoch may already be overfitted. Irvin et al. (2019) and Chowdhury et al. (2020) both pair augmentation with regularization when fine-tuning deep networks on imbalanced disease

labels.

Class imbalance is common in pneumonia datasets, where positive cases can outnumber normal ones in some splits. For both sigmoid and softmax outputs it is usually handled by augmenting the minority class, splitting with stratification, and early stopping, instead of simply maximizing accuracy. Accuracy is misleading on imbalanced data: a model that always predicts the majority class can score well while missing every minority case. We apply these techniques to all three models, with augmentation settings adjusted to each model's input size and architecture. For example, DenseNet121 uses `ImageDataGenerator` with rotation and shift settings, while EfficientNetB3 has `RandomZoom` and `RandomRotation` layers built into the Keras model.

2.7 Multi-Disease Detection Frameworks

Far fewer studies build multi-disease systems than single-disease ones. There are many papers on COVID-19 detection, TB screening, or pneumonia classification, each reporting good accuracy on its own. Most of these tools handle one disease and report offline metrics, with no connection to a clinical workflow. A clinician who suspected several possible causes would need to run separate applications, each with its own interface, preprocessing, and output format. In emergency triage, where speed and simplicity count, that is impractical.

CheXpert and CheXNet predict many conditions at once from a single model trained on all of them. That approach needs very large labelled datasets, and it returns all predictions together instead of screening for a particular suspected condition. Clinical triage often works from a differential diagnosis: a clinician may suspect TB where it is endemic, COVID-19 during an outbreak, or community-acquired pneumonia in a child.

Our system addresses this by training three architectures (EfficientNetB3, DenseNet121, and a custom CNN) on one four-class dataset (Normal, Tuberculosis, COVID-19, Pneumonia), so that the comparison is fair and every model predicts the same diseases. Each backbone keeps its own input size and preprocessing (DenseNet 224×224 , EfficientNet 256×256 , custom CNN 150×150), but all share the same splits, seed, augmentation budget, and four-class softmax output, and the two transfer models share the same classification head. A Flask application runs every available model on each upload and combines their probabilities by soft voting. It shows where the models agree and disagree, and the user does not have to choose a disease model first. Published medical AI work rarely does this: most studies either report single-disease notebooks or train one large multi-label network on a dataset such as CheXpert, without a controlled comparison of backbones.

2.8 Deployment and Clinical Translation

A common criticism of medical AI research is the “deployment gap”: many studies report accuracy, sensitivity, and AUC on retrospective datasets but never deliver software that clinicians

can use. TensorFlow (Abadi et al., 2015) and its ecosystem made it practical to save models and run inference at scale, which lowered the engineering barrier. Even so, peer-reviewed chest X-ray studies often stop at a notebook, reporting test-set metrics without showing that a non-expert can run the model reliably through a stable interface. A model with 95% accuracy in a controlled experiment gives no diagnostic value if it cannot be loaded, fed correctly preprocessed images, and run consistently in a clinic or community setting.

A deployable system needs engineering beyond training. It must check uploads in several stages and reject images that are not radiographs. Its inference preprocessing must match training exactly, including bilinear resizing where a library would default to nearest-neighbour. It should load large weight files only when needed so that the server starts quickly, and it must handle concurrent requests safely. OpenCV (Bradski, 2000) is the standard library for reading and checking images, and we use it in a three-stage chest X-ray gate (monochrome check, radiograph envelope features, and an anatomy veto) that runs before inference.

Lightweight Flask applications have become a common way for student and research prototypes to deploy medical AI. Our Flask application has an upload endpoint that accepts an image, validates it, runs `predict_all()` over every trained four-class model, and shows each model's probability bars alongside a soft-voting ensemble, without asking the user to choose a disease model. Inference runs on the server, so model weights and TensorFlow stay there. Regulatory-grade clinical deployment is a separate matter from research prototypes, but a working interface still makes an academic project more credible.

Our Flask application, which runs all three four-class models and presents their ensemble, is our response to the deployment gap described in Section 2.10.

2.9 Explainability and Trust

Clinicians will only adopt AI screening tools they can understand and trust, and accuracy alone does not earn that. A clinician given a prediction with no explanation is unlikely to act on it during triage, especially if it contradicts their own impression. Selvaraju et al. (2017) introduced Grad-CAM (Gradient-weighted Class Activation Mapping), which produces a heatmap of the image regions that most influenced a CNN's prediction. Grad-CAM takes the gradient of the target class score with respect to the activations of a chosen convolutional layer, giving a coarse localization map that can be laid over the X-ray.

Grad-CAM and similar saliency methods have been used on chest X-ray classifiers to check that models look at plausible anatomy, such as the lung fields, and not at dataset artefacts such as borders or text. If a model's heatmap lights up a hospital name printed on the border instead of the lung opacities, it has learned a shortcut that accuracy figures would never reveal. Despite wide interest, few multi-model screening systems with a single interface include explainability, and most explainability studies cover one model and one disease. Adding Grad-CAM or a similar

visual explanation to a multi-disease system is an open opportunity, and we list it as future work in the gaps below and in Chapter 6.

2.10 Gaps in Existing Research

Deep learning for chest X-rays has advanced a great deal, but few results have become multi-disease screening tools that clinicians can use. Many studies report strong accuracy on curated public datasets, and few deliver integrated software that a clinician could use during triage. Each gap below shaped the design, architecture, and deployment work described in Chapters 3 to 5. Closing them takes better models, but also attention to software integration, validation discipline, and deployment, which the literature covers far less than new architectures.

1. **Incomparable single-disease models:** Separate student/demo classifiers on incompatible label spaces prevent fair architecture comparison and fragment the user workflow.
2. **Leakage-unaware splits:** Patient overlap and duplicate files across train and test inflate reported accuracy.
3. **Accuracy-only reporting:** Headline accuracy hides weak minority-class recall (especially scarce COVID-19 samples).
4. **Train-serve skew:** Different resize or normalisation at inference versus training silently degrades live predictions.
5. **Weak input validation:** Monochrome-only checks accept many non-radiograph uploads.
6. **Missing ensemble transparency:** Few demos surface multi-model disagreement alongside an aggregated verdict.

2.11 Summary

The literature supports CNNs and ImageNet transfer learning for chest X-ray analysis. It also warns that small COVID datasets, dataset shortcuts, and reporting only accuracy weaken any claims made. Trustworthy results need clean data (patient-grouped splits and deduplication), matched training for a fair backbone comparison, and a deployable interface that states clearly it is not for diagnosis. Our project responds with a four-class benchmark across three architectures, evaluation that checks for leakage with a source probe, and a Flask ensemble application with a tested radiograph gate. Grad-CAM-style explainability is left for future work.

Table 2.1: Selected related works versus this project's focus

Work	Task	Dataset style	Method	Limitation vs this FYP
CheXNet (2017)	Multi-label CXR	Large NIH corpus	DenseNet121	Different label formulation; not a 4-class student demo
COVID-Net (2020)	COVID multi-class	Custom CXR set	Custom CNN	Single architecture; pandemic-era bias risks
Kaggle pneumonia CNNs	Binary pneumonia	Mooney/RSNA-style	Shallow / transfer CNNs	Binary only; often leakage-unaware student splits
Binary TB transfer studies	TB vs Normal	Public TB CXR	VGG / Efficient-Net	Not comparable across diseases
This project	4-class single-label	Merged public sources	3 architectures + ensemble	Public small data; research demo only

Chapter 3

Methodology

This chapter describes how we built and evaluated the system. Besides classifying chest X-rays, the methodology had to let us compare three deep learning architectures fairly and reproducibly, so all three are trained under the same conditions. The pipeline has six stages: merging the datasets, preprocessing and augmentation, model design, training, evaluation with a leakage check, and deployment in Flask with ensemble inference.

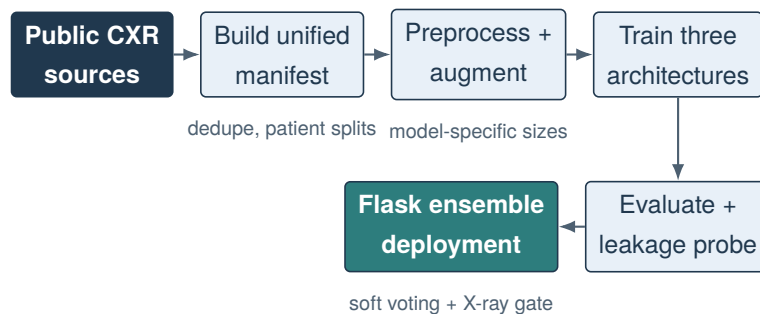


Figure 3.1: End-to-end methodology pipeline for the unified four-class system

3.1 From Single-Disease Models to a Unified Benchmark

The first version of this project used three separate models with incompatible label sets: binary tuberculosis, multi-class COVID-19 screening (which at the time included Viral Pneumonia), and binary pneumonia. We could not compare them properly, since a difference in performance might come from the datasets and say nothing about the architectures. We therefore reframed the problem as a single four-class task and trained all three architectures on the same merged dataset, with the same patient-grouped splits, the same classification head for the transfer models, and the same training budget. The class order is fixed as Normal, Tuberculosis, COVID-19, Pneumonia. Images labelled Viral Pneumonia in the COVID sources are counted as Pneumonia, because the Kaggle pneumonia dataset mixes bacterial and viral cases and there is no way to separate them after the fact.

3.2 Data Collection and Unified Manifest

We did not collect any new clinical data. A builder script (`training/build_manifest.py`) merges three public datasets into `training/manifest.csv`, which records each image's path, class, patient identifier, source dataset, and split (train, validation, or test).

3.2.1 Source Datasets

Table 3.1: Source datasets used to construct the unified four-class corpus

Source dataset	Expected path	Classes contributed
TB Chest X-Ray (Kaggle)	TB_Chest_Radiograph_Database/	Tuberculosis, Normal
Covid19-dataset	DenseNet121/Covid19-dataset/	COVID-19, Normal, Pneumonia
Chest X-Ray Pneumonia (Kaggle)	pneumonia_detection/data/chest_xray/	Pneumonia, Normal

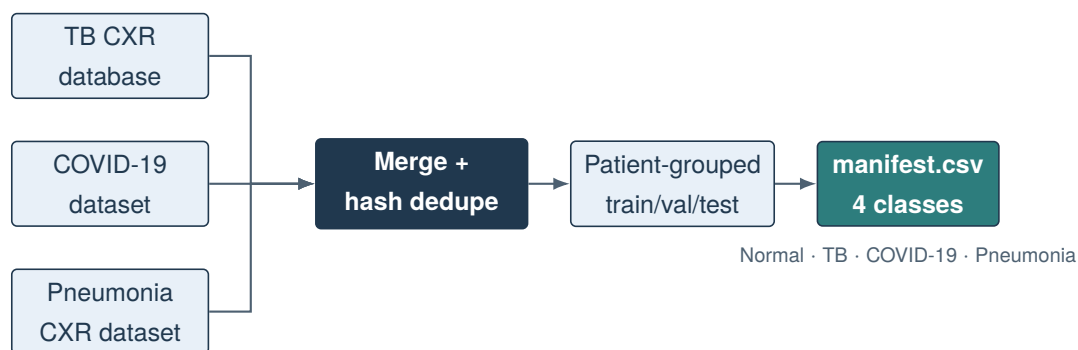


Figure 3.2: Dataset unification into a shared four-class training manifest

3.2.2 Design Rules for Honest Splits

Six rules keep the splits honest:

1. **Content-hash deduplication.** We remove byte-identical images, so the same file cannot end up in both the training and test sets.
2. **Patient-grouped splitting.** All images from one patient go into the same split, whether that is train, validation, or test.
3. **Fully qualified patient IDs.** Bare file names such as `0101.jpeg` repeat across the COVID folders, so we qualify each identifier with its path and group patients across the whole dataset.
4. **Multi-source Normal and Pneumonia.** Both classes draw on more than one dataset, which makes it harder for a model to tell classes apart by scanner or dataset style.
5. **Class-size capping with class weights.** COVID-19 has only 130 images after deduplication. We capped the other classes at 1,200 in the reported run (`-cap 1200`) and handled the remaining imbalance with inverse-frequency class weights (COVID-19 weight ≈ 2.851).
6. **Class×source inspection.** Before training, we check a class-by-source table to confirm that Normal and Pneumonia really do come from several datasets.

Every image comes from a public research dataset. The system is a research demonstration and a benchmark of architectures; it has not been clinically validated and is not a diagnostic device.

3.3 Preprocessing and Data Augmentation

3.3.1 Model-Specific Preprocessing

Table 3.2: Preprocessing per architecture

Model	Input size	Preprocessing
EfficientNetB3	$256 \times 256 \times 3$	EfficientNet preprocess_input
DenseNet121	$224 \times 224 \times 3$	DenseNet preprocess_input
Custom CNN	$150 \times 150 \times 3$	Divide by 255

Training and inference decode and resize images the same way, with bilinear interpolation. An earlier version resized with nearest-neighbour at inference but bilinear during training. That mismatch (train–serve skew) changed a large fraction of pixel values, so we fixed it.

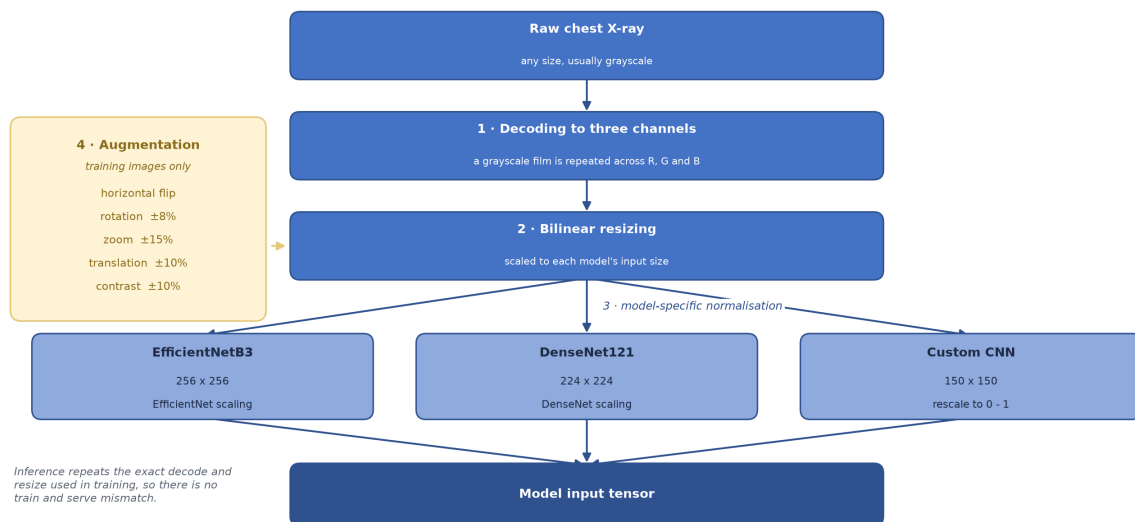


Figure 3.3: Preprocessing and augmentation pipeline with model-specific input sizes and normalisation

3.3.2 Augmentation and Class Imbalance

All three architectures get the same geometric augmentation from the shared data pipeline: rotation, translation, zoom, and horizontal flips, each within a set range. We did not oversample the disease images. Instead, inverse-frequency class weights in the categorical cross-entropy loss make mistakes on the smaller classes cost more.

3.4 Model Architectures and Shared Head

The models are defined in `training/models.py`. Both transfer models use the same classification head, so any difference between them comes from the backbone:

$\text{GAP} \rightarrow \text{Dropout}(0.5) \rightarrow \text{Dense}(256, \text{ReLU}) \rightarrow \text{Dropout}(0.3) \rightarrow \text{Dense}(4, \text{softmax})$.

Every model ends in a softmax layer trained with categorical cross-entropy, which fixes an earlier bug where a sigmoid output was paired with that loss. None of the models start from the old single-disease weights.

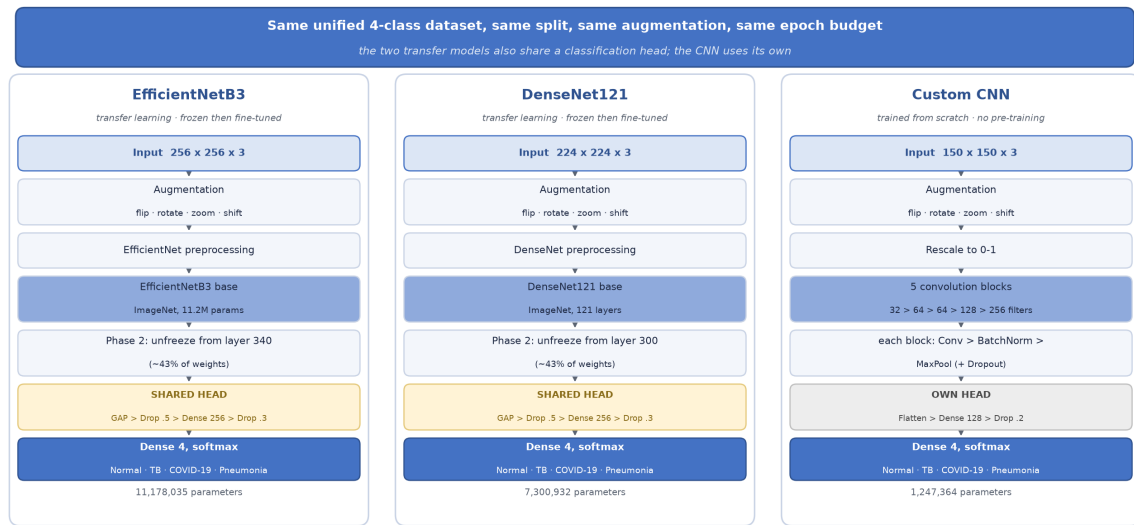


Figure 3.4: Three architectures scored under a matched four-class protocol (shared head for transfer models)

EfficientNetB3 and DenseNet121 both start from ImageNet weights. In Phase 2, EfficientNetB3 is unfrozen from layer 340 onward (~42.7% of its parameters) and DenseNet121 from layer 300 (~42.8%), and DenseNet121's BatchNorm layers stay frozen during fine-tuning. The custom CNN is our from-scratch baseline: five convolutional blocks (filters $32 \rightarrow 64 \rightarrow 64 \rightarrow 128 \rightarrow 256$) with BatchNorm, pooling, and dropout, ending in a four-class softmax head.

3.5 Training Protocol

Table 3.3: Shared training hyperparameters and controls

Setting	Value
Shared seed	1337
Batch size	32 (identical across models when resources allow)
Phase-1 learning rate	1×10^{-4} (Adam), backbone frozen
Phase-2 learning rate	1×10^{-5} (Adam), partial unfreeze for transfer models
Loss	Categorical cross-entropy with class weights
Output	4-way softmax
Test set	Never used for training or model selection
Artefacts	<code>weights/*_4class.keras</code> , <code>training/runs/*.json</code>

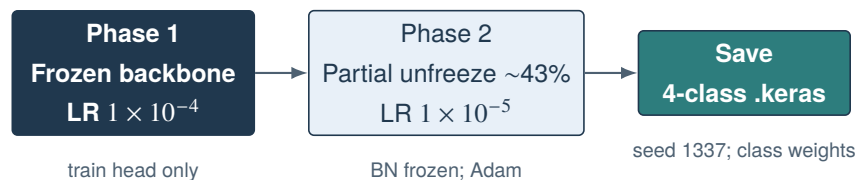


Figure 3.5: Two-phase transfer-learning protocol used for EfficientNetB3 and DenseNet121

Training uses checkpointing, early stopping, and ReduceLROnPlateau callbacks. One command trains all three models: `python training/train.py -model all`.

3.6 Evaluation Methodology

The script `training/evaluate.py` scores every model on the held-out test split. It reports accuracy, balanced accuracy (the mean of per-class recall), per-class precision, recall, and F1, confusion matrices, a comparison across architectures, and soft-voting ensemble metrics, and writes them to `training/results/comparison.md`. Soft voting averages the models' probability vectors and picks the class with the highest average. Unlike a hard majority vote, it keeps track of how confident each model was.

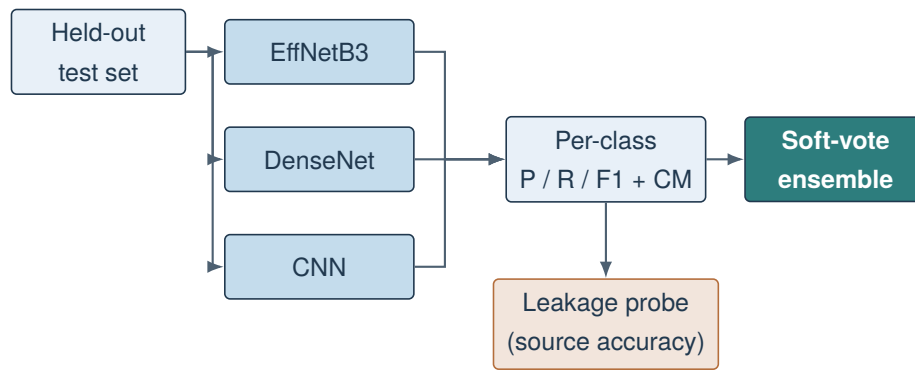


Figure 3.6: Evaluation flow: matched test scoring, ensemble soft voting, and source-leakage probe

The leakage probe (`training/leakage_probe.py`) trains a classifier to predict which dataset each image came from and compares it with a majority-class baseline. Ideally it does little better than the baseline. If it could name the source almost perfectly, a model might be recognising TB or COVID-19 from how the images were acquired instead of from the disease itself. COVID-19 recall is the weakest point of the evaluation, because the class has so few test images.

3.7 Application Integration Workflow

In the application, the user uploads a PNG, JPG, or JPEG file of up to 16 MB. The image goes through the three-stage chest X-ray gate, unless the user chooses Analyse anyway. `predict_all()` then runs every model on it, the soft-voting ensemble combines their outputs, and the results page shows each model's probability bars, the verdict, and whether the models agree. Weight files must output four classes, and legacy `.h5` files are rejected. To reproduce the whole system, run `build_manifest.py`, then `train.py --modelall`, then `evaluate.py`, optionally `leakage_probe.py`, and finally `Frontend/app.py` to start the application.

3.8 Methodological Validity and Limitations

The comparison is valid because the task is clearly defined, leakage is controlled, the architectures are trained the same way, the metrics include per-class recall, and deployment uses the same preprocessing as training. It has known limits. The public datasets are small, and COVID-19 recall is statistically unstable. Four-class accuracy will be lower than the old binary figures. TB and COVID-19 each come from a single source, so a model could learn source cues instead of disease (which is why we run the leakage probe). The system is not a clinical device, and Grad-CAM and multi-centre validation are left for future work.

3.9 Summary

We replaced three separate detectors with one four-class benchmark in which three architectures share the same data, splits, and training protocol, with leakage controls and ensemble deployment. Because the data and protocol are shared, differences between the models can be attributed to the architectures. The evaluation also states plainly what the datasets cannot show and that the system is not for diagnosis.

Chapter 4

System Design

This chapter describes the software built on that methodology. It has five layers: the presentation layer (HTML, CSS, and Jinja templates), the application layer (Flask routes), an inference service that loads models on first use, guards them with a lock, and computes the ensemble, the model files in `weights/`, and an offline training pipeline in `training/`.

4.1 System Architecture

When a user uploads an image, a Flask route receives it and the chest X-ray gate checks it. `predict_all()` in `predictors.py` then scores it with EfficientNetB3, DenseNet121, and the custom CNN, and the soft-voting ensemble combines their outputs. Finally, the results page shows the probability bars and whether the models agree.

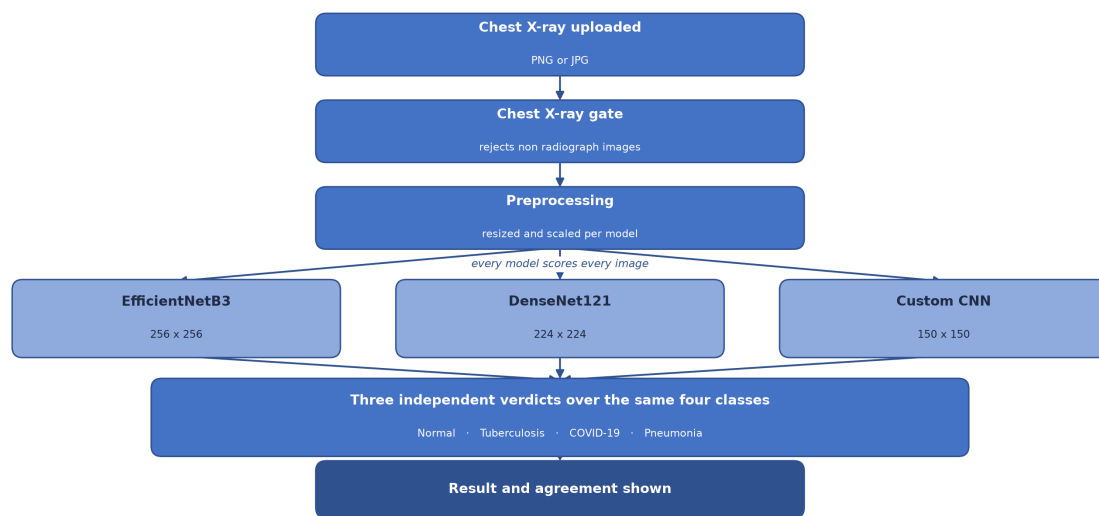


Figure 4.1: High-level inference flow for the unified four-class web application

Every trained four-class model scores every upload, always with the class order Normal, Tuberculosis, COVID-19, Pneumonia, and soft voting averages their probability vectors. The server refuses old single-disease `.h5` files whose output is not four classes wide. If a weight file is missing, it reports the problem and keeps running.

Table 4.1: Model integration overview

Architecture	Output	Input	Interface
EfficientNetB3	4-class softmax	256×256	Flask ensemble
DenseNet121	4-class softmax	224×224	Flask ensemble
Custom CNN	4-class softmax	150×150	Flask ensemble

4.2 Functional Requirements

- **FR1:** Accept PNG/JPG/JPEG uploads (maximum 16 MB).

- **FR2:** Validate likely chest radiographs; allow forced override with warning.
- **FR3:** Run every available 4-class model on the same image.
- **FR4:** Display per-class probabilities per model.
- **FR5:** Compute and display soft-voting ensemble and agreement.
- **FR6:** Show clear errors if weights are missing or have wrong output width.
- **FR7:** Provide offline training scripts to rebuild weights from the manifest.

4.3 Non-Functional Requirements

- **NFR1 Reproducibility:** Fixed seed and committed manifest policy.
- **NFR2 Usability:** Single upload; no model picker required.
- **NFR3 Robustness:** Multi-stage rejection of non-X-ray uploads.
- **NFR4 Consistency:** Train/serve preprocessing alignment.
- **NFR5 Maintainability:** Shared CLASSES order in `training/common.py`.
- **NFR6 Safety messaging:** Explicit research-only / non-clinical disclaimer.

4.4 Tools and Technologies Used

The system is written in Python (3.9 or later; 3.11 is recommended for current TensorFlow). It uses TensorFlow and Keras, OpenCV, Flask and Werkzeug, NumPy, scikit-learn for evaluation, Matplotlib and Seaborn for figures, HTML and CSS, and Git. Training is best done on a machine with a capable GPU. Inference also runs on a CPU, only more slowly. The main folders are `training/`, `Frontend/`, and `weights/`, along with the three public dataset folders described in Chapter 3.

4.5 Chest X-Ray Validation Gate

Before any model sees an upload, the gate checks it in three stages:

1. **Monochrome or tint check.** The image must be grayscale or a single tint. Blue-tinted display captures are allowed.
2. **Radiograph envelope.** Statistics such as intensity, entropy, midtones, aspect ratio, and symmetry must look like those of a real radiograph. One violation is forgiven.
3. **Anatomy veto.** The image is rejected only if it lacks both the bright central-column cues and the left-right symmetry cues.

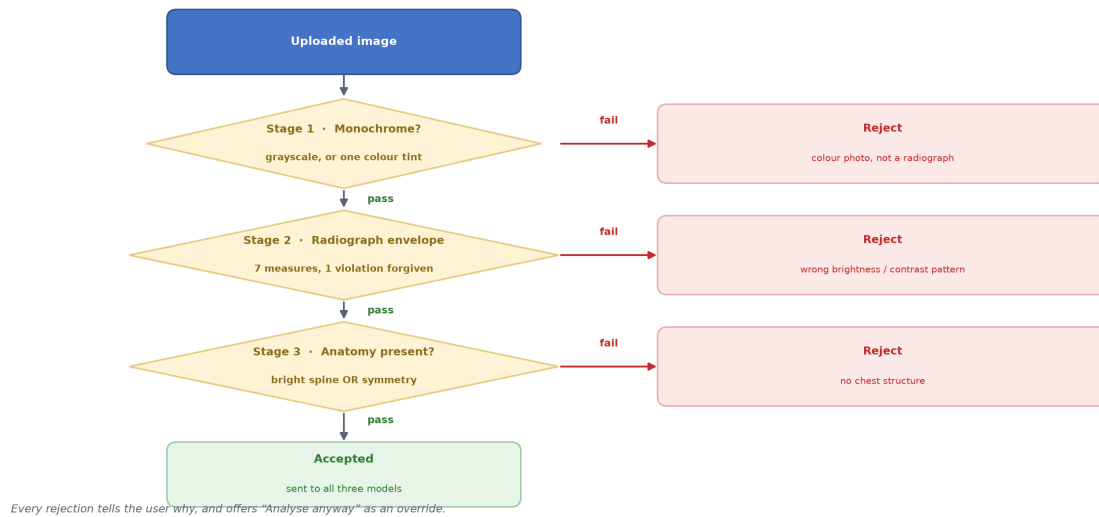


Figure 4.2: Three-stage chest X-ray validation gate with optional forced analysis

We tested the gate on a labelled set of 351 real radiographs and 106 other images. It accepted all of the real radiographs and rejected about 77% of the non-X-rays, where the earlier monochrome-only check rejected only about 21%. When the gate rejects an image it tells the user why, and the Analyse anyway option is protected against path traversal.

4.6 Application Flow and User Interface

The home page shows which models are ready and holds the upload form. The results page shows each model's probability bars, a short description of each class, the ensemble verdict, whether the models agree, and a warning when the user forced the analysis past the gate. Figures 4.3 and 4.4 show the upload page and a gate rejection. Chapter 5 shows result pages for TB, COVID-19, and pneumonia examples on which all three models agreed.

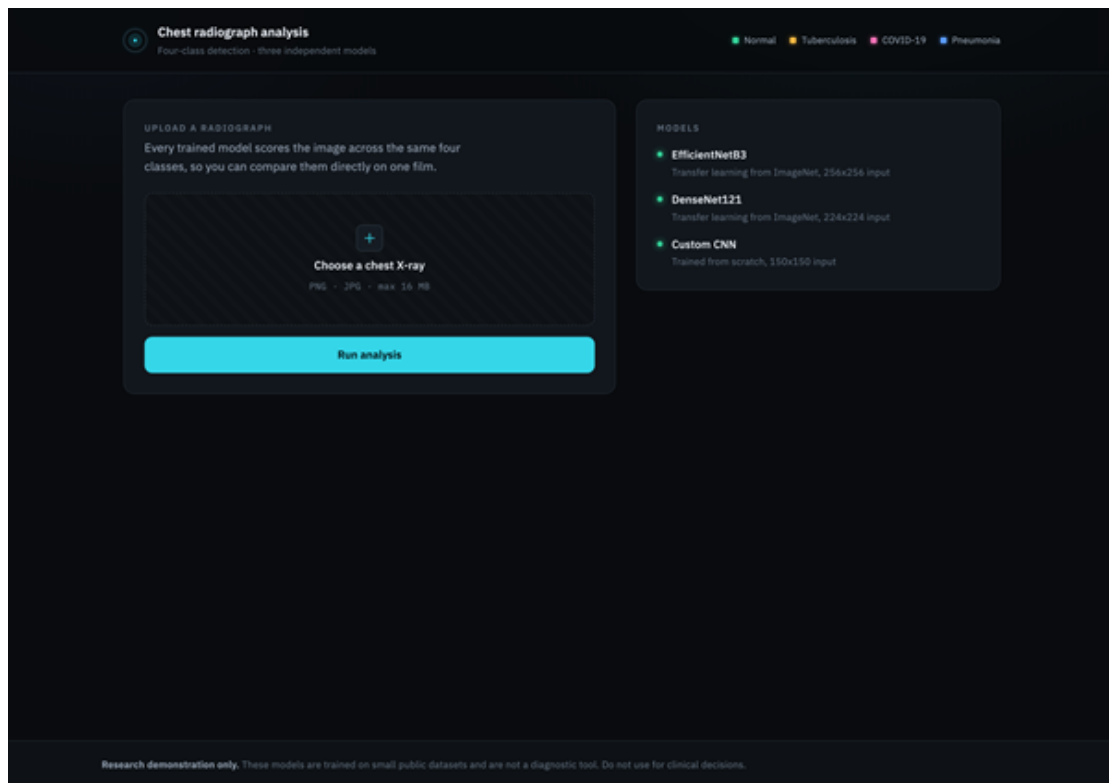


Figure 4.3: Flask upload interface for four-class multi-model chest radiograph analysis

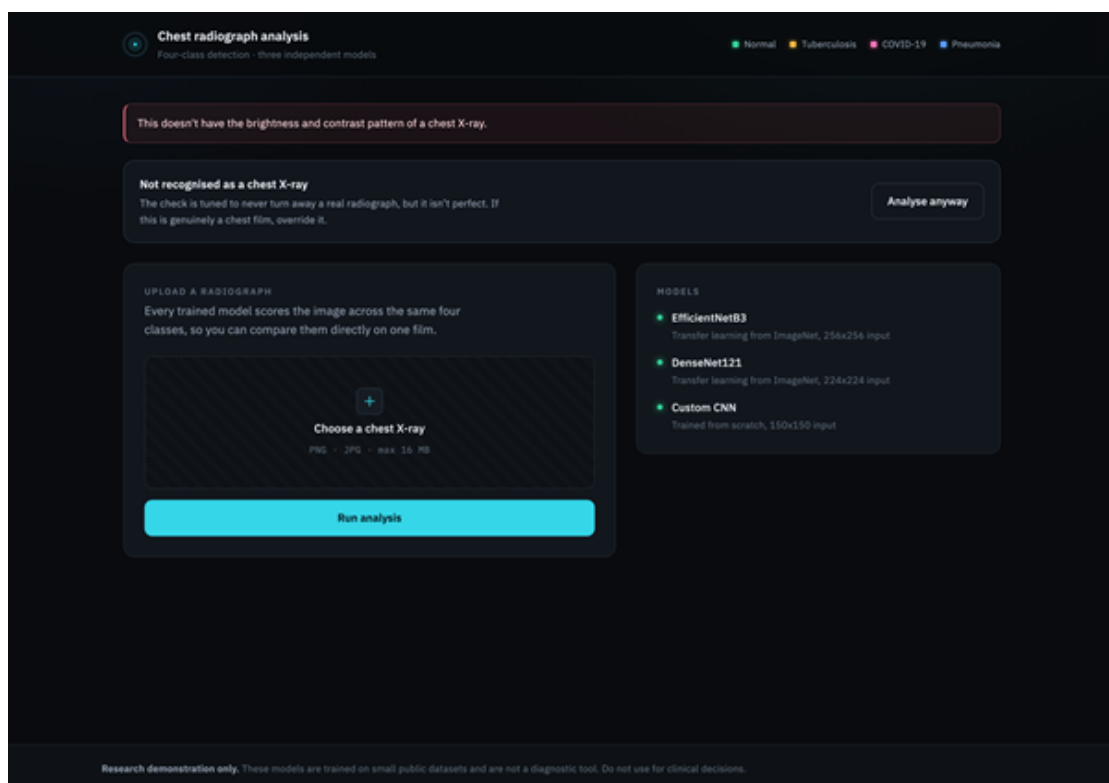


Figure 4.4: Chest X-ray validation gate rejection with Analyse anyway option

Table 4.2: Flask application components

Component	Location	Purpose
Web Framework	Flask	Routes and templates
Inference	Frontend/predictors.py	Gate, lazy load, ensemble
Shared constants	training/common.py	Class order, preprocess
Weights	weights/*.keras	4-class models

4.7 Train/Serve Consistency

At inference, the application decodes and resizes images exactly as training did, with bilinear interpolation and the same `preprocess_input` or `/255` rescaling. Training, evaluation, and the user interface all take the class order from one shared `CLASSES` list, and the application refuses any weight file that does not output four classes.

4.8 Summary

The system turns the methodology into a research demo that people can use. It checks uploads before they reach the models, scores every image with all three models, combines them by soft voting, and preprocesses images the same way in the live application as in training.

Chapter 5

Results and Discussion

This chapter reports how the four-class system performed on the held-out test split of `training/manifest.csv`. Every accuracy, precision, recall, F1, confusion-matrix, and leakage-probe figure comes from the evaluation exports (`training/results/comparison.md`, `results.json`, `ensemble.json`, `leakage_probe.json`) and the run logs. Scores from the earlier single-disease models appear only in their own section, labelled as prior work, and never in the four-class tables.

5.1 Experimental Setup

We used seed 1337, patient-grouped splits, and the class order Normal, Tuberculosis, COVID-19, Pneumonia. The splits came out at 2,260 training, 485 validation, and 482 test images (70.0%, 15.0%, and 14.9%). All models were trained with a batch size of 32 and inverse-frequency class weights (COVID-19 weight ≈ 2.851), and each class was capped at 1,200 images for this run (`-cap 1200`).

Table 5.1: Training environment (machine that produced the four-class weights)

Item	Value
OS	macOS 15.7.9 (x86_64)
CPU	Intel Core i9-9980HK @ 2.40 GHz (16 logical cores)
Memory	32 GB
GPU	None (CPU-only TensorFlow)
Python / TensorFlow / Keras	3.11.15 / 2.16.2 / 3.15.0
Training dates	DenseNet121: 4 Aug 2026; EfficientNetB3: 5 Aug 2026; CNN: 9 Aug 2026
Evaluation	Per-model comparison: 9 Aug 2026; ensemble + leakage: 13 Sep 2026

5.2 Unified Manifest Summary

Removing duplicates by MD5 content hash took out 45 images and left a pool of 10,328. COVID-19 dropped from 137 to 130 images. The manifest selected from this pool has 3,227 images. The COVID source’s Viral Pneumonia images are counted as Pneumonia, which is where the 89 `covid_db` images in that class come from.

Table 5.2: Selected unified manifest counts by class and split

Class	Train	Val	Test	Total
Normal	840	180	180	1,200
Tuberculosis	488	105	104	697
COVID-19	91	20	19	130
Pneumonia	841	180	179	1,200
Total	2,260	485	482	3,227

Table 5.3: Class $\times$ source contingency (selected manifest)

Class	covid_db	pneumonia_db	tb_db	Total
Normal	88	556	556	1,200
Tuberculosis	0	0	697	697
COVID-19	130	0	0	130
Pneumonia	89	1,111	0	1,200
Total	307	1,667	1,253	3,227

Normal and Pneumonia draw on several datasets by design. Tuberculosis and COVID-19 each come from a single dataset, so a model could more easily learn to recognise them by their source (Section 5.6).

5.3 Four-Class Comparative Results

All models were scored on the same test split ($n = 482$). The ensemble figures come from the offline `ensemble.json`, which averages the three models' class probabilities.

Table 5.4: Overall four-class test metrics ($n = 482$)

Model	Accuracy	Bal. Acc.	Macro P	Macro R	Macro F1
EfficientNetB3	82.99%	83.00%	0.767	0.830	0.745
DenseNet121	90.87%	89.14%	0.850	0.891	0.864
Custom CNN	90.25%	88.36%	0.813	0.884	0.835
Ensemble (soft vote)	92.32%	93.07%	0.842	0.931	0.867

DenseNet121 is the best single model on accuracy (90.87%), balanced accuracy (89.14%), and macro F1 (0.864). EfficientNetB3 has the highest COVID-19 recall of the three (89.47%), but its COVID-19 precision is much lower. Combining the models lifts accuracy to 92.32% and balanced accuracy to 93.07%.

We treat balanced accuracy as the main metric. COVID-19 has only 19 test images, so plain accuracy would barely register how well the models handle it.

5.4 Per-Class Precision, Recall, and F1

5.4.1 Recall summary

Table 5.5: Per-class recall on the test set

Model	Normal	TB	COVID-19	Pneumonia
EfficientNetB3	78.3%	69.2%	89.5%	95.0%
DenseNet121	86.1%	88.5%	84.2%	97.8%
Custom CNN	80.6%	96.2%	78.9%	97.8%
Ensemble	85.6%	94.2%	94.7%	97.8%

5.4.2 DenseNet121 (best single-model macro F1)

Table 5.6: DenseNet121 per-class metrics

Class	Precision	Recall	F1	Support
Normal	92.3%	86.1%	0.891	180
Tuberculosis	97.9%	88.5%	0.929	104
COVID-19	59.3%	84.2%	0.696	19
Pneumonia	90.7%	97.8%	0.941	179

5.4.3 Ensemble (soft voting)

Table 5.7: Ensemble per-class metrics

Class	Precision	Recall	F1	Support
Normal	95.7%	85.6%	0.903	180
Tuberculosis	97.0%	94.2%	0.956	104
COVID-19	48.7%	94.7%	0.643	19
Pneumonia	95.6%	97.8%	0.967	179

COVID-19 precision is low for every model, and no single model gets above about 60%. The ensemble's COVID-19 recall of 94.7% looks strong, but it rests on only 19 test cases ($n = 19$) and could shift noticeably on a different sample, so it should always be quoted with that caveat.

5.5 Confusion Matrices

In each matrix the rows are the true classes and the columns the predicted classes, both in the order Normal, Tuberculosis, COVID-19, Pneumonia.

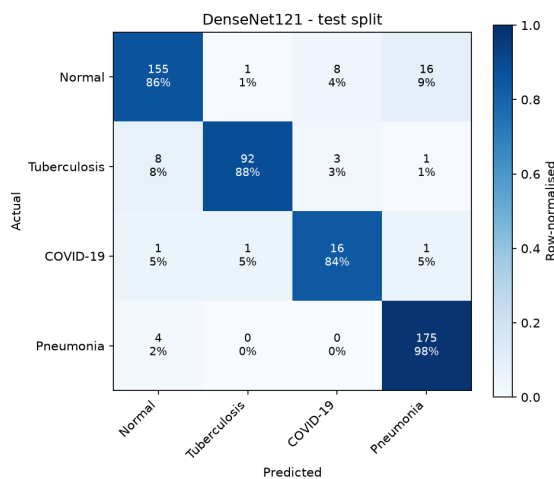


Figure 5.1: DenseNet121 confusion matrix

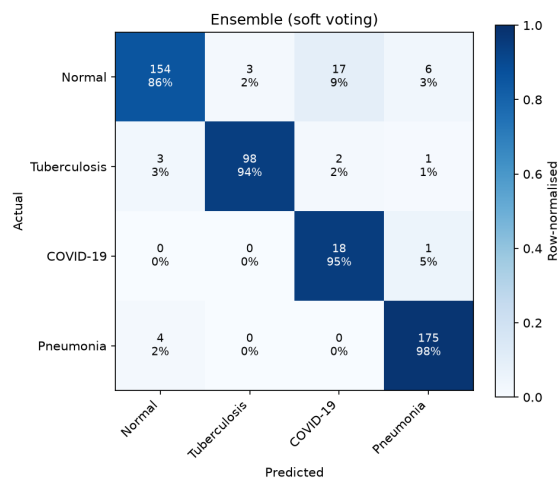


Figure 5.2: Ensemble confusion matrix

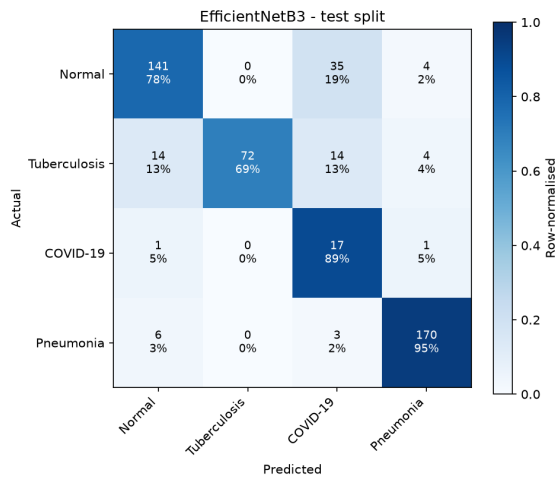


Figure 5.3: EfficientNetB3 confusion matrix

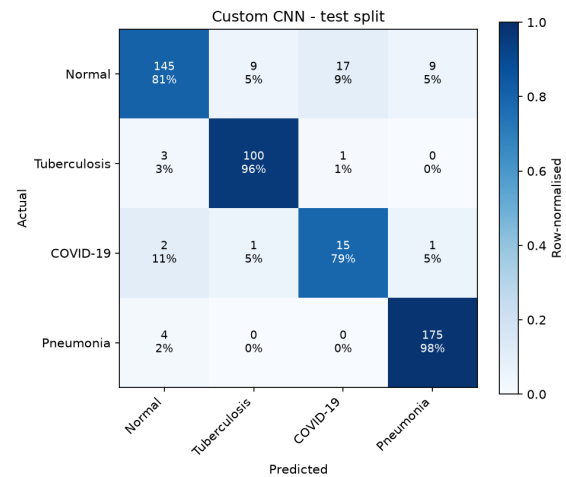


Figure 5.4: Custom CNN confusion matrix

5.6 Leakage Probe

The probe is a custom CNN at 150×150 , trained for 8 epochs to predict which dataset each image came from. It named the source correctly 88.17% of the time, against a majority-class baseline of 49.79%, a gap of 38.4 percentage points. We rate this as a moderate source signal: the datasets can partly be told apart by source, though far from perfectly. Because Tuberculosis and COVID-19 each come from a single dataset, part of their recall may rest on scanner or processing cues. Normal and Pneumonia come from several datasets and are less exposed to that shortcut. The disease results earlier in this chapter should be read with this caveat in mind.

5.7 Training Run Summary

Table 5.8: Saved validation metrics and training cost

Model	Val Acc.	Val Loss	Epochs	Train time
EfficientNetB3	80.00%	0.492	20	~35.7 h (CPU)
DenseNet121	89.90%	0.299	20	310.6 min
Custom CNN	88.87%	0.354	20	36.8 min

The models have roughly 11.2 M (EfficientNetB3), 7.3 M (DenseNet121), and 1.25 M (custom CNN) parameters. EfficientNetB3 took so long to train because the Mac we used had no GPU, so it trained on the CPU alone.

5.8 Deployed Interface Demonstrations

Figures 5.5 to 5.7 show the Flask interface returning soft-voting verdicts on which all three models agree, with each model's probability bars. The application combines the models with the same soft-voting rule used in the offline evaluation above.

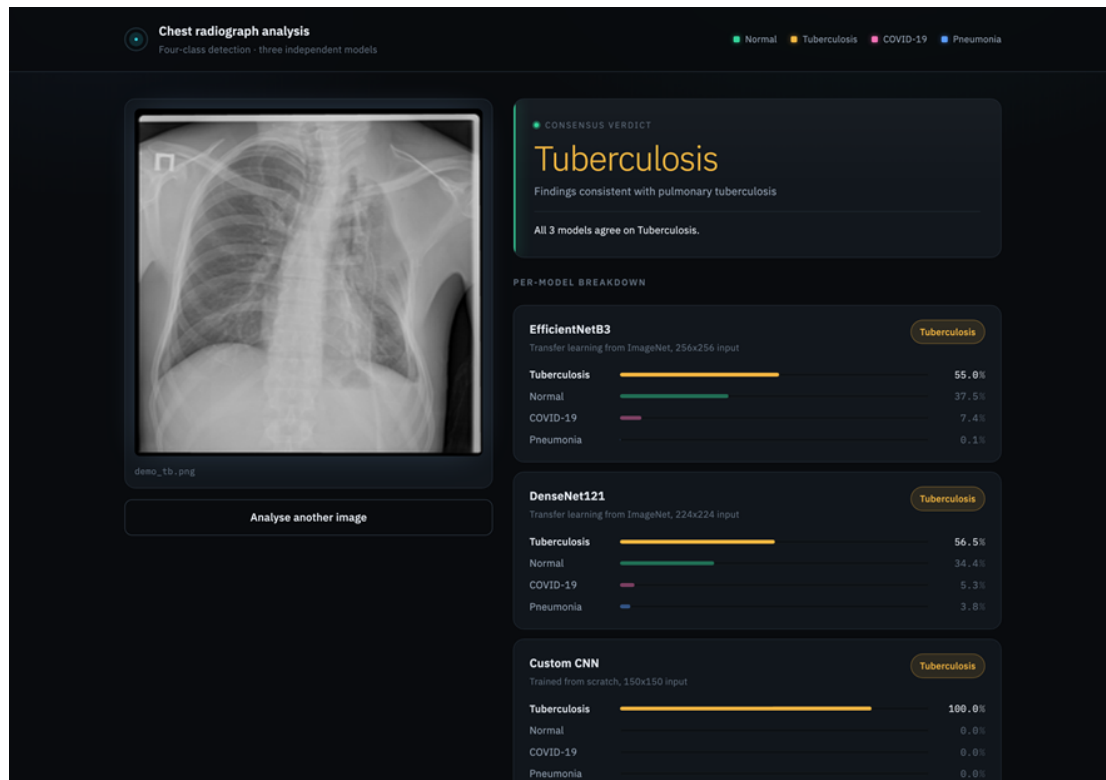


Figure 5.5: Consensus Tuberculosis verdict with per-model 4-class probability bars

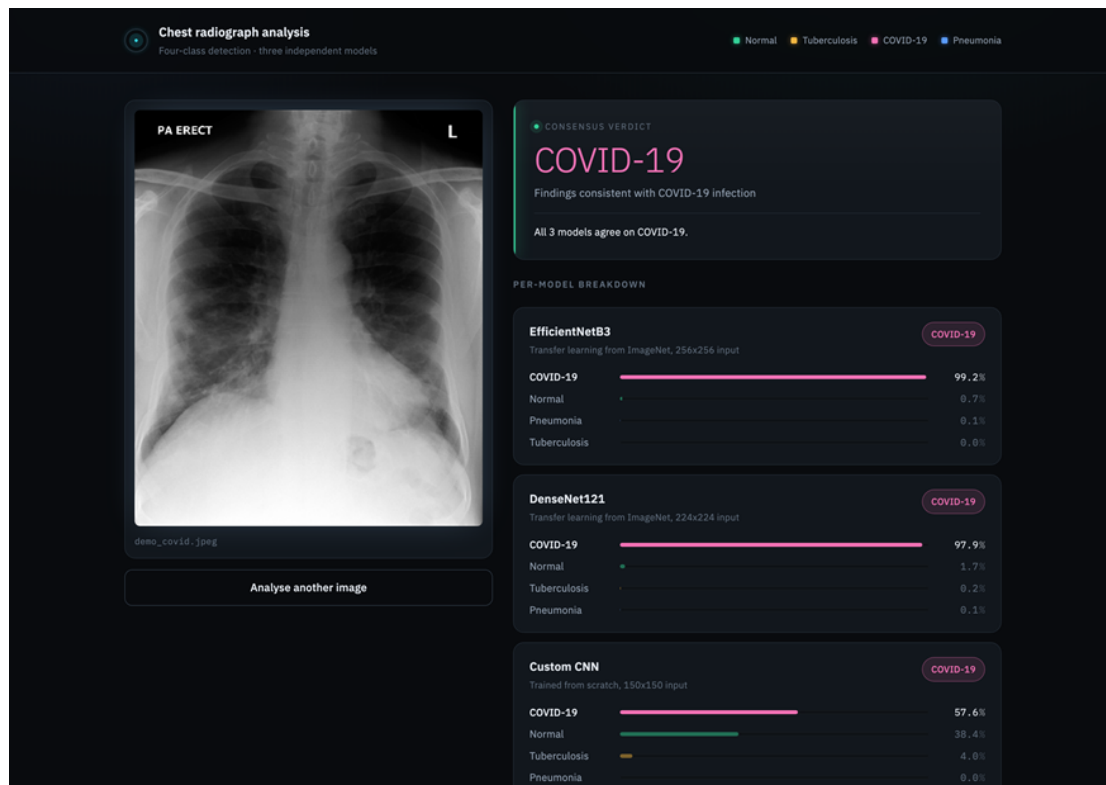


Figure 5.6: Consensus COVID-19 verdict with per-model 4-class probability bars

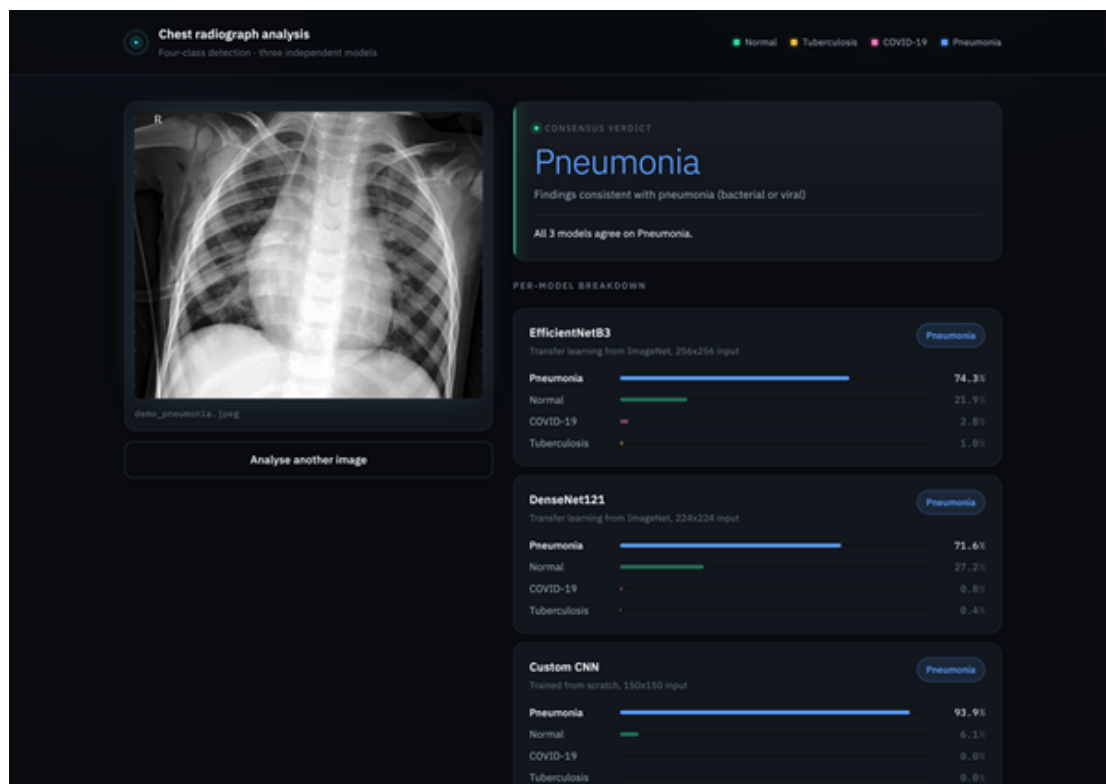


Figure 5.7: Consensus Pneumonia verdict with per-model 4-class probability bars

5.9 Challenges Faced and Solutions Applied

Table 5.9: Challenges and solutions during the 4-class conversion

#	Challenge	Solution
1	Incomparable old models	Unified 4-class manifest
2	Patient/filename leakage	Grouped IDs + hash dedupe
3	Sigmoid + categorical bug	Softmax heads for all models
4	Unfair fine-tune by layer index	Match ~43% unfreeze fraction
5	Non-X-ray uploads	Rebuilt 3-stage gate (21%→77% reject)
6	Train/serve resize skew	Align bilinear decode
7	Missing P/R/F1 historically	<code>evaluate.py</code> suite
8	Small COVID set ($n=130$)	Class weights + honest recall/precision reporting

5.10 Historical Single-Disease Work

Our earlier binary models scored 95.63% test accuracy for TB (EfficientNetB3) and 92% for pneumonia (custom CNN, on 624 test images). The old COVID-19 model overfitted so badly that its validation accuracy was near 0%. These figures are **not comparable** with the four-class results in this chapter, and we include them only as background on how the project developed.

5.11 Implementation Status

Table 5.10: Implementation status after evaluation export

Component	State
Unified manifest	Complete (3,227 images; Table 5.2)
Four-class weights	Present
Held-out evaluation	Complete (Tables 5.4–5.7)
Leakage probe	Complete (88.17% vs 49.79% baseline)
Frontend ensemble UI + gate	Complete

5.12 Summary of Findings

On the four-class test set ($n = 482$), DenseNet121 is the strongest single backbone (90.87% accuracy, 89.14% balanced accuracy, macro F1 0.864), and the soft-voting ensemble does better still, with 92.32% accuracy and 93.07% balanced accuracy. Every model recognises pneumonia well, with recall of at least 95%. COVID-19 is the hard class. Its recall can be high (94.7% for the ensemble) while precision stays low, and with only 19 COVID-19 test images neither figure is stable. The leakage probe found a moderate source signal (88.17% against a 49.79% baseline), so the TB and COVID-19 results, which each come from one dataset, need cautious reading. The redesign met its goal: the three architectures were compared fairly on shared data, and the per-class metrics show how the minority class performs, which overall accuracy would have hidden.

Chapter 6

Future Recommendations

6.1 Conclusion

We set out to build a research-grade chest X-ray system that sorts images into four classes (Normal, Tuberculosis, COVID-19, Pneumonia), benchmarks EfficientNetB3, DenseNet121, and a custom CNN on the same data, and serves all three through a Flask application with an ensemble. The project met that aim. On the held-out test set ($n = 482$), DenseNet121 was the strongest single model (90.87% accuracy, 89.14% balanced accuracy), and the soft-voting ensemble reached 92.32% accuracy and 93.07% balanced accuracy. The comparison between models holds up because they share one manifest and patient-grouped splits, duplicates were removed by content hash, each backbone was fine-tuned to the same fraction, results are reported per class, and a leakage probe (88.17% vs 49.79% baseline) measured how much the data source gives away. The system remains a research demonstration on public data and is not a clinical diagnostic device.

6.2 Recommendations

- Always report per-class recall for minority diseases such as COVID-19.
- Keep patient-grouped splits in any dataset extension.
- Do not compare old binary metrics to four-class metrics without clear historical labelling.
- Use the leakage probe whenever heterogeneous public sets are merged.

6.3 Future Enhancements

- Additional verified COVID data and multi-centre validation.
- Grad-CAM or related explainability overlays.
- DICOM support and PACS-oriented integration paths.
- Calibration (e.g., temperature scaling) and per-class decision thresholds.
- Active learning or semi-supervised expansion.
- Mobile/cloud deployment with authentication and audit logs.
- Optional additional backbones (e.g., VGG19) under the same harness.

6.4 Summary of Contributions

1. A unified four-class CXR benchmark from three public sources with leakage controls.
2. A fair three-architecture training harness (shared head; matched fine-tune fraction).
3. An evaluation suite with precision/recall/F1, balanced accuracy, ensemble scoring, and a leakage probe.
4. A Flask multi-model demo with soft voting, agreement display, and a measured X-ray gate.
5. Documentation enabling reproducibility (manifest, scripts, and status notes).

Bibliography

- [1] Rajpurkar, P., Irvin, J., Ball, R. L., et al. (2017). CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning. *arXiv:1711.05225*.
- [2] Huang, G., Liu, Z., Van Der Maaten, L., & Weinberger, K. Q. (2017). Densely Connected Convolutional Networks. *CVPR 2017*.
- [3] Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. *ICML 2019*.
- [4] Wang, L., Lin, Z. Q., & Wong, A. (2020). COVID-Net. *Scientific Reports*, 10(1), 19549.
- [5] Chowdhury, M. E. H., Rahman, T., Khandakar, A., et al. (2020). Can AI Help in Screening Viral and COVID-19 Pneumonia? *IEEE Access*, 8, 132665–132676.
- [6] Lakhani, P., & Sundaram, B. (2017). Deep Learning at Chest Radiography: Automated Classification of Pulmonary Tuberculosis. *Radiology*, 284(2), 574–582.
- [7] Irvin, J., Rajpurkar, P., Ko, M., et al. (2019). CheXpert. *AAAI 2019*.
- [8] Mooney, P. (2018). Chest X-Ray Images (Pneumonia). Kaggle. <https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia>
- [9] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. *CVPR 2016*.
- [10] Raghu, M., Zhang, C., Kleinberg, J., & Bengio, S. (2019). Transfusion: Understanding Transfer Learning for Medical Imaging. *NeurIPS* 32.
- [11] Rahman, T., Khandakar, A., Qiblawey, Y., et al. (2021). Exploring Image Enhancement for COVID-19 Detection. *Computers in Biology and Medicine*, 132, 104319.
- [12] Jaeger, S., Candemir, S., Antani, S., et al. (2014). Two Public Chest X-Ray Datasets for Computer-Aided Screening of Pulmonary Diseases. *QIMS*, 4(6), 475–477.
- [13] Hwang, S., Kim, H. E., Jeong, J., & Kim, H. J. (2016). A Novel Approach for Tuberculosis Screening Based on Deep CNNs. *SPIE Medical Imaging 2016*.
- [14] Simonyan, K., & Zisserman, A. (2014). Very Deep Convolutional Networks for Large-Scale Image Recognition. *arXiv:1409.1556*.
- [15] World Health Organization. (2022). *Global Tuberculosis Report 2022*. Geneva: WHO.
- [16] Abadi, M., et al. (2015). TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems.
- [17] Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*.
- [18] Selvaraju, R. R., Cogswell, M., Das, A., et al. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. *ICCV 2017*.
- [19] Zhang, J., Xie, Y., Wu, Q., & Xia, Y. (2020). Medical Image Classification Using Synergic Deep Learning. *Medical Image Analysis*, 54, 10–19.
- [20] Deng, J., Dong, W., Socher, R., et al. (2009). ImageNet: A Large-Scale Hierarchical Image

- Database. *CVPR 2009*.
- [21] Wang, X., Peng, Y., Lu, L., et al. (2017). ChestX-ray8: Hospital-Scale Chest X-Ray Database and Benchmarks. *CVPR 2017*.
 - [22] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet Classification with Deep Convolutional Neural Networks. *NeurIPS 2012*.
 - [23] Grinberg, M. (2018). *Flask Web Development*, 2nd ed. O'Reilly Media.
 - [24] Rahman, T., et al. Tuberculosis (TB) Chest X-ray Database. Kaggle. <https://www.kaggle.com/datasets/tawsifurrahman/tuberculosis-tb-chest-xray-dataset>
 - [25] World Health Organization. (2023). *Global Tuberculosis Report 2023*. Geneva: WHO.

Chapter A

Dataset Details

A.1 Dataset Overview

The dataset has four mutually exclusive classes: Normal, Tuberculosis, COVID-19, and Pneumonia. In the reported run, COVID-19 has 130 images after deduplication by content hash, Normal and Pneumonia are capped at 1,200 images each, and training uses inverse-frequency class weights.

A.1.1 Data Sources

Table A.1: Dataset sources, paths, and known public links

Dataset	Local path	Supplies	Public link
TB Chest X-Ray	TB_Chest_Radiography_Database/	TB, Normal	Kaggle TB CXR corpus
Covid19-dataset	DenseNet121/Covid19-dataset/	COVID-19, Normal, Pneumonia	Local COVID folder (317 images on checkout)
Chest X-Ray Pneumonia	pneumonia_detection/data/chest_xray/	Pneumonia, Normal	Kaggle CXR pneumonia corpus

The manifest used for evaluation has 3,227 images: 2,260 for training, 485 for validation, and 482 for testing, with 130 COVID-19 images after deduplication. Viral Pneumonia images from the COVID source are counted as Pneumonia. Chapter 5 gives the full split and class-by-source tables.

A.1.2 Preprocessing and Splits

Chapter 3 describes the patient-grouped splits (seed 1337), deduplication by content hash, the shared geometric augmentation, and each architecture’s input size and preprocessing.

Chapter B

Model Architecture Summaries

B.1 Model Training and Implementation

- EfficientNetB3 / DenseNet121: ImageNet base plus shared head (GAP, Dropout, Dense 256, Dropout, Softmax 4); Phase 1 frozen; Phase 2 unfreeze from layers 340 / 300 (~43%).
- Custom CNN: five-block stack ending in Softmax(4).
- Scripts: `training/build_manifest.py`, `common.py`, `models.py`, `train.py`, `evaluate.py`, `leakage_probe.py`; frontend under `Frontend/`; weights `efficientnetb3_4class.keras`, `densenet121_4class.keras`, `cnn_4class.keras`.
- Run order: build manifest, then train all, then evaluate, optional leakage probe, then `Frontend/app.py`. A portable inference bundle under `xray-classifier-portable/` already ships the three *.keras weight files for demo without retraining on the local machine.

Chapter C

Alignment with Sustainable Development Goals (SDGs)

Table C.1: Potential educational and research alignment with SDGs (non-clinical)

No.	SDG	Contribution language
1	SDG 3 Good Health and Well-being	Decision-support <i>research</i> for respiratory screening accessibility, with explicit non-diagnostic limits.
2	SDG 4 Quality Education	Reproducible teaching artefact for medical AI methodology (fair splits, metrics literacy).
3	SDG 9 Industry, Innovation and Infrastructure	Open software pipeline for multi-model CXR benchmarking.
4	SDG 10 Reduced Inequalities	Potential relevance to resource-limited screening contexts as <i>future clinical research</i> ; not a deployed medical device.

We keep these claims modest, since this FYP is an educational research demonstration built on public data.